Bitirme projesi için case araştırması

Soru: Bu proje hangi problemi çözüyor? Problemi hangi avantajla çözüyoruz ve mevcut çözümlerden farkımız ne?

Yöntem: Dört araştırma hattı yürütüldü: deterministik sıkıştırma, CAN bus anomali tespiti, sertifikasyon, ve rakip platformlarla diğer adaylar. Kaynaklar Ekim 2026'da arXiv, DOI, standart ve proje sayfalarından doğrulandı. Doğrulanamayan iddialar **(belirsiz)** diye işaretli. "Bulunamadı" bir arama sonucudur, yokluğun kanıtı değildir.

---

0. Kısa cevap

Önerilen ana case: Uydu ve edge cihazlarında, deterministik bir dil modeliyle kayıpsız telemetri ve log sıkıştırma.

Bir dil modeli bir sonraki byte'a olasılık verir. Arithmetic coding bu olasılığı bit'e çevirir. Bunun çalışması için encoder ile decoder'ın **bit bit aynı** olasılığı hesaplaması gerekir. Floating point ile çalışan modeller bunu farklı donanımlarda garanti edemez. Bizim integer ve bit-exact engine'imiz ise bunu tasarımı gereği sağlar:

- encoder FPGA'de çalışır: CPU yok, OS yok, gecikme sabit;
- decoder yerde, aynı integer programı çalıştıran C ya da Python kodudur.

Yan case: Aynı modelle anomali tespiti. Bir byte'ın sıkıştırma maliyeti $-\log_2 p$ bit'tir. Bu değer aynı zamanda bir anomali skorudur: model bir olaya ne kadar şaşırıyorsa, onu kodlamak o kadar çok bit tutar. Yani tek bir model ve tek bir hardware aynı veriyi hem sıkıştırır hem de izler. Uygulama alanı olarak iki seçenek var:

- **Telemetri** (varsayılan). Ana case ile aynı veri; ek iş az.
- **Araç içi CAN bus.** Regülasyonu ve veri setleri güçlü, ama rekabet de güçlü.

Çapraz gereksinim: sertifikalanabilir ve tekrarlanabilir inference. Kendi başına bir case değil; iki case'in de kanıt paketine eklenir. EASA'nın AI rehberi FPGA'yi mevcut sertifikasyon yolunun içinde sayıyor, GPU'yu ise kapsam dışında bırakıyor.

---

1. Elimizdeki sistemin güçlü ve zayıf yanları

Case seçimi, sistemin gerçekten neyi iyi yaptığına dayanmalı. Ölçümler aşağıda.

1.1 Hız: aynı model, farklı platformlar (stories260K)

Platform	Clock	token/s	Token başına cycle	Kaynak
Apple M1, 1 core, run.c -Ofast	3,2 GHz	14 397	~222 bin	`model/zoo/cpu_baseline.json`

Platform	Clock	token/s	Token başına cycle	Kaynak
Bu proje, XC7A100T (simülasyon)	25 MHz	**496**	**50 397**	`model/zoo/nexys.json`
Siracusa RISC-V + NPU (16 nm silicon), benzer INT8 Llama	360 MHz	340	–	arXiv 2408.04413
STM32H7A3 (Cortex-M7), FP32	280 MHz	87	~3,2 M	github.com/schuhandreas/embedllama-stm32h7a3 (README'ye göre)
ESP32-S3, FP32	240 MHz	19–33	~7–12 M	github.com/DaveBben/esp32-llm ve diğer portlar (README'ye göre)
HLSTransform, VU9P (Llama2 110M; model farklı)	200–300 MHz	57	–	arXiv 2405.00738 (host gerekli)

Tablodan çıkanlar:

- **Ham hızda üstün değiliz.** M1 bizden 29 kat hızlı, çünkü clock'u 128 kat yüksek.
- **Cycle başına verimliyiz.** Token başına M1'den 4,4 kat, ESP32-S3'ten 140–240 kat daha az cycle harcıyoruz.
- **MCU'lardan hızlıyız.** ESP32-S3'ten 15–26 kat, STM32H7'den 5,7 kat.

1.2 Enerji: üstünlük iddia edemiyoruz

- Çip düzeyinde tahminimiz ~1 200–3 300 token/J. XC7A100T'nin statik gücü ~0,24 W (DS181 Table 5, 85 °C); dinamik güç tahmin.
- Siracusa'nın ölçülmüş değeri 2 040 token/J. M1 için tahmin ~1 000–3 000 token/J.
- Kart düzeyinde (~2 W) avantaj tamamen kayboluyor.
- Statik güç yüzünden always-on, pilli cihazlar bu FPGA için uygun bir hedef değil.

1.3 Gerçekten ayırt edici olan özellikler

1. **Bit-exact determinizm.** Aynı integer program RTL'de, netlist'te, Python'da (`IntRunner`) ve C'de (`sw/c/zoo_run.c`) birebir aynı sonucu veriyor. 18 modelde 11 241 token'da fark 0.
2. **Host, CPU, OS ve DRAM yok.** Paylaşılan cache, interrupt ya da scheduler yok. Bu yüzden cache tabanlı side-channel saldırıları yapı gereği uygulanamıyor.
3. **Veriden bağımsız, önceden hesaplanabilen latency.** Cycle sayısı token pozisyonunun bir formülü: stories260K'da t = 0'da 38 652, t = 511'de 79 532 cycle.

4. **Küçük model, hızlı yeniden eğitim.** 50K parametrelik modeller dakikalar içinde yeni veride eğitiliyor ve birkaç RAMB36'ya sığıyor.
5. **Kalite bedeli küçük.** Integer'a geçiş loss'u names modellerinde en fazla %0,3, stories260K'da %1,6 artırıyor.

Sonuç: Doğru case, bu özelliklerden en az birinin **işlevsel bir zorunluluk** olduğu, hız ya da enerjinin ise birincil ölçüt olmadığı bir problem olmalı.

---

2. Değerlendirilen case'ler

Puanlar 1–5 arası. Sütunlar:

- **Gerçeklik:** problem gerçek ve belgelenmiş mi?
- **Zorunluluk:** bizim özelliklerimiz şart mı, yoksa yalnızca güzel mi?
- **Fark:** mevcut çözümlerden ne kadar ayrışıyoruz?
- **Sim:** sonuç yalnız simülasyonla kanıtlanabilir mi?
- **İş:** ek iş az mı? (yüksek puan = az iş)

Case	Gerçeklik	Zorunluluk	Fark	Sim	İş	Genel
C1 · Deterministik LM sıkıştırma (telemetri, log)	4	**5**	4	**5**	4	**4/5, ana case**
C2 · LM ile anomali tespiti (telemetri ya da CAN bus)	**5**	4	3	4	3	**4/5, yan case**
C3 · Sertifikalanabilir AI (DO-254, EASA, ISO 26262)	4	3	4	3	3	Bağımsız 2/5; çapraz gereksinim olarak 4/5
A · Tekrarlanabilir ve doğrulanabilir inference	4	4	3	5	5	4/5; C1'in kanıtına eklenir
B · Açık eğitim ve referans tasarımı	3	2	4	5	5	Tez çerçevesi olarak 4/5, endüstriyel case olarak 3/5
Side-channel'sız inference	3	3	3	3	3	3/5; ileri çalışma
Network flow classification	4	2	1	4	2	2/5; 1 Gbps için 2–3 mertebe yavaşız
Always-on metin üretimi	3	1	1	4	4	1/5; statik güç

C1 neden ana case? Determinizm burada bir avantaj değil, sistemin çalışmasının ön koşulu. Ayrıca bütün iddia simülasyonda uçtan uca kanıtlanabiliyor: RTL sıkıştırır, Mac'teki C kodu açar ve byte'lar

birebir aynı çıkar. Bu, kart kullanmama kararımızla tam uyumlu.

---

3. Ana case: deterministik dil modeliyle kayıpsız sıkıştırma

3.1 Sezgi ve sayısal örnek

- Model bir sonraki byte'a %90 olasılık veriyorsa o byte'ı kodlamak $-\log_2 0{,}9 \approx 0{,}15$ bit tutar; %1 olasılık veriyorsa 6,6 bit. Ortalama maliyet modelin cross-entropy'sidir. Sonuç: modeli iyi olan veriyi iyi sıkıştırır (Delétang vd., ICLR 2024, arXiv 2309.10668).
- Arithmetic coding [0, 1) aralığını olasılıklara göre böler. Encoder p("a") = 0,30000001, decoder 0,29999999 hesaplarsa aralık sınırları farklılaşır. Kodlanan sayı tam bu iki sınırın arasına düşerse decoder başka bir sembol seçer. Ondan sonraki **bütün** byte'lar bozulur.
- Gerçek sayılarla: 2048 byte'lık bağımsız paketlerde gzip enwik9'u %48,1'e, 200K parametrelik bir transformer %30,9'a indiriyor (Delétang vd., Table 1). Yani paket başına ~%36 daha az byte. Bizim modellerimiz de 50K–260K parametre aralığında.

3.2 Problem ve kanıtları

Determinizm sorunu belgelenmiş:

- Ballé, Johnston ve Minnen (ICLR 2019, "Integer networks for data compression with latent-variable models"): platforma bağlı floating point round-off yüzünden range coding'in "can fail catastrophically" olduğunu yazıyor; çözüm olarak integer networks öneriyor.
- He vd. (arXiv 2202.07513): non-deterministic hesap olasılıkları "cross-platform inconsistent" yapıyor.
- llama-zip (github.com/AlexBuz/llama-zip, README): çıktı "may not be decompressible on a different system, even if the same model is used". Aynı modelde bile `--n-gpu-layers` değerinin eşleşmesi gerekiyor.
- NNCP v2 (Bellard, 2021): PyTorch'un deterministic modu yalnız "exact same hardware and software versions" altında geçerli.
- Armstrong vd. 2026 (arXiv 2603.25526) buna "GPU Butterfly Effect" diyor. Adler ve Tang 2026 (arXiv 2601.10678) bu uyumsuzluğa dayanıklı yeni bir coder öneriyor. Sorun güncel.

Uzayda ihtiyaç var: CCSDS 124.0-B-1 (POCKET+, Şubat 2023) gerekçesinde üç şey yazıyor:

- housekeeping parametre sayısı bant genişliğinden hızlı artıyor;
- downlink "fire and forget" çalıştığı için "zip or tar unsuitable";
- flight processor'lar zayıf.

Yani hem sıkıştırma hem de paket kaybına dayanıklılık (her paketin bağımsız açılabilmesi) şart.

3.3 Mevcut çözümler ve sınırları

Çözüm	Ne yapıyor	Sınırı
gzip, zstd, xz	Sözlük ve pencere tabanlı	Küçük bağımsız paketlerde oran çok düşüyor (gzip 2048 B'de %32,3'ten %48,1'e)

Çözüm	Ne yapıyor	Sınırı
CCSDS 121.0-B-3 (Rice, libaec)	Basit predictor ve Golomb-Rice; FPGA'de 205 Msamples/s	Predictor basit; metin ve log için zayıf
CCSDS 124.0-B-1 (POCKET+)	Housekeeping paketlerinde XOR/mask; oran 4–16, OPS-SAT'ta tipik 8–10	Yalnız sabit uzunluklu paketler; metin ve log için tasarlanmamış; FPGA uygulaması bulunamadı
NNCP, cmix	En iyi oranlar (enwik9'da ~0,85 bpb)	Float; GPU ya da 31 GB RAM; NNCP kodlarken eğitim yapıyor
ts_zip, llama-zip, LLMZip	LLM ile çok iyi oran	GPU; decoder ancak aynı donanımda garanti
L3TC (arXiv 2412.16642)	200K'lık RWKV; enwik9'da %18,5 (zstd -22 %21,5)	Float; mobil ve GPU hedefli
DRAC (Zynq-7000)	Delta-RNN ve arithmetic coding	Fixed point olup olmadığı ve determinizmi **(belirsiz)**
RAS (arXiv 2511.04684)	Bit-exact rANS coder	Yalnız coder; modeli içermiyor

3.4 Bizim çözümümüz

`FPGA (uydu ya da edge cihazı) Yer istasyonu (Mac, PC) byte → LM (zoo_core) → integer frekans tablosu paket → range decoder ← integer frekans tablosu → range coder → paket ←——— aynı integer program (zoo_run.c / IntRunner) (bağımsız; raw escape bayrağı)` Bileşenler:

- **Var:** engine, integer referans ve C ISS (`sw/c/zoo_run.c`). Yer tarafındaki decoder'ın temeli hazır.
- **Yeni (küçük):**
- integer softmax çıktısından frekans tablosu: her değer ≥ 1, toplam 2^k;
- 32-bit range coder;
- paketleyici ve domain dışı veri için "raw escape" bayrağı.

Ek yükü token başına ~256 cycle tahmin ediyorum (ölçülmedi). Bu, mevcut 6–50 bin cycle'ın %5'inin altında.

3.5 Farkımız

Özellik	gzip / zstd	POCKET+	Float neural (NNCP, ts_zip, llama-zip)	Biz
Küçük paketlerde iyi oran	hayır	yalnız HK paketlerinde	evet	evet (doğrulanacak)
Platformdan bağımsız decode	evet	evet	**hayır**	**evet, bit-exact**

Özellik	gzip / zstd	POCKET+	Float neural (NNCP, ts_zip, llama-zip)	Biz
CPU/OS gerekmez	hayır	hayır (flight CPU)	hayır (GPU)	**evet**
Sabit, önceden bilinen latency	hayır	kısmen	hayır	**evet**
Aynı modelle anomali skoru	hayır	hayır	mümkün	**evet**

"İlk" iddiası: Taramada şu üçünü birlikte yapan bir çalışma **bulunamadı**:

1. integer-only, bit-exact bir LM'i
2. arithmetic coder ile birlikte tamamen fabric'te çalıştırmak
3. sonucu başka bir platformda decode etmek.

3.6 Ölçülebilir başarı kriterleri

1. **Bit-exact round-trip:**
2. RTL simülasyonu, IntRunner, C/ARM (M1) ve C/x86 (Docker) arasında bütün test paketlerinde %100 aynı decode.
3. **Karşı deney:** aynı float modeli PyTorch CPU'da sıkıştırıp MPS'te açmak ve kaç paketin bozulduğunu saymak. Böylece sorunu kendi ortamımızda göstermiş oluruz; sıfır çıkarsa bunu da raporlarız.
4. **Oran:**
5. Aynı paketleme ile (bağımsız 256 B, 1 KB, 4 KB) bit/byte.
6. Karşılaştırma: gzip -9, zstd -19/-22, **eğitilmiş sözlüklü zstd** (küçük paketlerde adil rakip budur), xz -9e, CCSDS 121 (libaec), housekeeping için POCKET+.
7. **Hız:** coder dahil byte/s, paket gecikmesi ve 25/50 MHz'te LUT/BRAM kullanımı.
8. **Dayanıklılık:**
9. p oranında paket kaybında açılabilen paket oranı;
10. domain dışı girdide şişme paket başına en fazla 1 byte.

3.7 Veri setleri (açık)

- **Log ve metin:** Loghub (16'dan fazla sistem logu; github.com/logpai/loghub; arXiv 2008.06448). Tam dosyada gzip ~15×, lzma ~32× (LogShrink, arXiv 2309.09479).
- **Uzay telemetrisi:** ESA-ADB (arXiv 2406.17826), OPSSAT-AD (arXiv 2407.04730), NASA SMAP/MSL (arXiv 1802.04431). Bunlar parametre zaman serisi; sabit uzunluklu paketlere serileştirilmeleri gerekir.
- **Genel:** enwik8, enwik9, Canterbury.

3.8 Fizibilite ve riskler

Veri hızları (token = byte):

- Uydu housekeeping verisi 150–450 B/s (STEREO SECCHI, 1,2–3,6 kbps). CubeSat UHF linkleri 1 200–9 600 bit/s.
- Bizim mlp ve rnn modellerimiz ~7 400–7 700 B/s işliyor. Bu, housekeeping için 15–50 kat pay demek.
- 9 600 bit/s'lik bir link 4× oranla tamamen dolarsa ham veri 4,8 kB/s olur. Buna yalnız mlp ve rnn yetişir; GRU (3,8 kB/s) ve transformer'lar yetişmez.

Riskler:

Risk	Önlem
Housekeeping binary paketlerinde POCKET+ (8–10×) çok güçlü; geçtiğimize dair kanıt yok	Ana hedef log ve metin telemetrisi; HK ikincil deney
Model statik; misyon boyunca veri değişirse oran düşer	Model BRAM'de; yeni model = yeni BRAM içeriği; bunu sınır olarak yaz
Büyük dosyada xz/zstd'yi geçmek ≥800K parametre istiyor	Hedef küçük bağımsız paketler; büyük dosya iddiası yok
Mevcut modeller names ve TinyStories'te eğitildi	Byte düzeyinde yeniden eğitim (dakikalar)
`zoo_run.c` negatif sayıların aritmetik sağa kaydırılmasını varsayıyor	Derleyicide `static_assert` ve testler
Artix-7 rad-hard değil; bir BRAM SEU encoder'ı decoder'dan koparır	Sınır olarak yaz; ileri çalışma: BRAM ECC, CRC
Kart yok	İddialar simülasyon ve STA ile sınırlı; enerji yalnız Vivado tahmini

4. Yan case: aynı modelle anomali tespiti

Fikir: –log p hem kodlama maliyeti hem anomali skorudur. Model alışılmadık bir olay gördüğünde onu kodlamak çok bit tutar; bu da alarm sebebidir. Model yalnız normal veriyle eğitilir. Saldırı ya da arıza etiketi gerekmez, bu yüzden daha önce görülmemiş (zero-day) olayları da yakalayabilir.

4.1 Seçenek 1 (önerilen): telemetri anomali tespiti

- Ana case ile **aynı veri ve aynı model**. Ek iş yalnız skorlama ve eşik.
- Örnekler ve baseline'lar:
- Telemanom: LSTM, NASA SMAP/MSL, KDD 2018; yer segmentinde çalışıyor.
- OPS-SAT'ta on-board anomaly detection (ESA; accuracy 0,957, RandomForest).
- **Kriterler:** OPSSAT-AD ve ESA-ADB'de precision, recall ve F1, Telemanom ile aynı metrikle; INT8 modelin float modele göre en fazla ~2 puan düşük olması.

4.2 Seçenek 2: araç içi CAN bus saldırı tespiti

- **Problem gerçek ve regüle:** CAN'de authentication yok. UNECE R155 (madde 7.3.7) saldırıları "detect and prevent" etmeyi istiyor; AB'de 7 Temmuz 2024'ten beri bütün yeni araçlarda

zorunlu.

- **Veri:** ROAD (arXiv 2012.14600; masquerade dahil) ve can-train-and-test (arXiv 2308.04972; görülmemiş araç ve saldırı split'leri).
- **Rakipler:**
- CAN-BERT: ~3M parametre, GPU'da 0,8–3,8 ms.
- Desta vd.'nin next-ID LSTM'i: Xeon'da 24 ms.
- Shreejith grubunun Zynq tabanlı IDS'leri: ARM PS'te Linux çalışıyor, supervised.
- SecCAN (arXiv 2505.14924): CAN controller'ın içinde, PS'siz ve deterministik, ama supervised ve yalnız iki frame'e bakıyor.
- **Farkımız:**
- unsupervised ve autoregressive bir LM;
- aynı veri ve hardware üzerinde bigram'dan Llama tarzı modele kadar bir model ailesi karşılaştırması;
- frame içinde karar: model frame n'in dağılımını frame n−1 sırasında hesaplar, ID'nin 11 biti gelince skor bir tablo okuması olur.

Hız hesabı (500 kbit/s, tam bus load, 8 byte'lık frame, 4 505 frame/s):

Model	Gereken hızın kaç katı
bigram	71×
mlp	1,6×
rnn	1,7×
GRU	0,84× (yetişmez)
transformer'lar	0,5–0,6× (yetişmez)

Gerçek bir araçta ölçülen 680 frame/s'yi bütün modeller karşılıyor.

Zayıf yanlar:

- Yalnız ID dizisine bakan bir model masquerade'i göremez. Payload token'ları eklenirse hız ~9 kat düşer.
- Car-Hacking gibi kolay veri setlerinde her yöntem F1 ≈ 1 alıyor; fark ancak ROAD'da görülebilir.
- Gerçek bir CAN arayüzü yok; bit düzeyinde erişim için CTU CAN FD gibi ayrı bir IP gerekir.

Öneri: Telemetri seçeneği ana case ile bütünlük ve iş yükü açısından daha uygun. CAN seçeneği daha somut ve regülasyonla destekleniyor, ama rekabeti ve iş yükü daha büyük. Seçim hocayla birlikte yapılmalı.

---

5. Çapraz gereksinim: sertifikalanabilirlik ve tekrarlanabilirlik

Neden önemli?

- EASA AI Concept Paper Issue 2 (Mart 2024) şunları istiyor:
- inference modelinin latency, WCET ve bellek gereksinimleri (IMP-01);

- eğitilen model ile hedefte çalışan model arasındaki toleransın tanımlanması (IMP-01);
- platform farklarının analizi (IMP-07);
- formal methods (IMP-06: "expected to be necessary").

IMP-05 FPGA ve ASIC'leri mevcut rehberin (AMC 20-152A) kapsamında sayıyor. GPU'ları ise "very complex interference mechanisms or non-deterministic pipelining" gerekçesiyle kapsam dışı bırakıyor.

- Thinking Machines Lab (Eylül 2025): Qwen3-235B, temperature 0 ile 1 000 kez çalıştırıldığında 80 farklı çıktı verdi.
- Airbus (Geyer vd., arXiv 2401.08225): "an FPGA design comes as a white box being a clear advantage for certification."

Bizim sunduğumuz kanıtlar:

- Tolerans sıfır (bit-exact).
- WCET ölçümle değil, kapalı bir formülle veriliyor.
- Her operatörün range'i ispatlı.
- Synthesis araçlarında bulunan 3 bug. FAA'nın tool qualification raporu (DOT/FAA/TC-19/46), synthesis araçlarının qualification'ının pratikte yapılmadığını yazıyor; bizim bulgumuz neden bağımsız çıktı doğrulaması gerektiğini gösteriyor.

Dürüst sınır: Gerçek bir DO-254 sürecinden geçmedik. "Certified" ya da "DO-254 uyumlu" demek yanlış olur. Doğru ifade: "EASA ve DO-254 hedeflerine eşlenmiş kanıt".

Demo: C1'in round-trip deneyinin yanına üç şey eklenir:

- float modelin farklı batch, thread ve cihazlarda verdiği farklı çıktı sayısı;
- IMP-01...IMP-07 için bir gap analysis tablosu;
- requirement → RTL → test traceability matrisi.

---

6. Rota

Faz	Süre	İş	Çıktı	Karar noktası
0 · Yazılım fizibilitesi	1 hafta	Loghub ve ESA verisinde byte düzeyinde mlp, rnn ve küçük GPT eğitimi; IntRunner ile integer bit/byte; 256 B, 1 KB ve 4 KB paketlerde gzip, zstd-dict, xz ve POCKET+ ile kıyas	Oran tablosu	**Eğitilmiş sözlüklü zstd geçilemiyorsa** anomali tespiti ana case olur
1 · Bit-exact codec (yazılım)	1–2 hafta	Integer frekans tablosu ve range coder spec'i (Python); `zoo_run.c` tabanlı C decoder; M1 ve x86'da round-trip; float karşı deneyi (CPU ve MPS)	%100 round-trip; float'ta bozulan paket sayısı	–

Faz	Süre	İş	Çıktı	Karar noktası
2 · Hardware	2–3 hafta	Frekans tablosu op'u ve range coder unit'i; paketleyici; tam kart simülasyonunda sıkıştırma, Mac'te açma	Byte-exact decode; byte/s; LUT/BRAM; P&R	–
3 · Yan case	2 hafta	Aynı modelle anomali skoru; OPSSAT-AD ve ESA-ADB (ya da ROAD)	F1, AUROC, latency	–
4 · Kanıt ve yazım	1–2 hafta	IMP gap analysis, traceability; rapor ve sunum	Final rapor	–

Faz 0 yalnız yazılım ve birkaç günlük iş. Case'in tutup tutmadığını hardware'e girmeden gösterir.

---

7. Hocaya sunulabilecek problem tanımı

Problem. Uydu ve edge cihazları her gün artan miktarda telemetri ve log üretiyor, ama downlink bant genişliği dar ve paketler kaybolabiliyor. Dil modelleri bu tür verileri klasik yöntemlerden (gzip, zstd, CCSDS) daha iyi sıkıştırabiliyor. Ancak floating point ile çalıştıkları için encoder ile decoder farklı donanımlarda aynı olasılığı hesaplayamıyor ve veri açılamaz hale gelebiliyor. Ayrıca bu modeller GPU ya da güçlü bir CPU istiyor.

Çözümümüz. Karpathy'nin dil modeli ailesini CPU'suz bir FPGA'de yalnız integer arithmetic ile, bit-exact ve sabit latency ile çalıştıran bir engine tasarladık. Aynı integer program yer istasyonunda C ve Python ile birebir aynı sonucu veriyor. Bu sayede model, küçük ve bağımsız paketlerde kayıpsız sıkıştırma yapabiliyor ve sıkıştırdığı veri her platformda açılabiliyor. Aynı modelin verdiği olasılık, anomali skoru olarak telemetriyi izlemek için de kullanılıyor.

Farkımız. Klasik sıkıştırıcılar küçük paketlerde zayıf. Neural sıkıştırıcılar ise platforma bağımlı ve büyük donanım istiyor. Bizim çözümümüz ikisinin arasında: neural kalite, klasik algoritmaların determinizmi, CPU'suz ve sertifikasyona uygun bir hardware.

---

8. Kaynaklar (seçme)

Sıkıştırma ve determinizm

- Delétang vd., "Language Modeling Is Compression", ICLR 2024, arXiv 2309.10668
- Ballé, Johnston, Minnen, "Integer networks for data compression with latent-variable models", ICLR 2019, openreview S1zz2i0cY7
- He vd., arXiv 2202.07513
- Armstrong vd. 2026, arXiv 2603.25526
- Adler ve Tang 2026, arXiv 2601.10678
- Bellard, NNCP (bellard.org/nncp) ve ts_zip

- llama-zip (github.com/AlexBuz/llama-zip)
- L3TC, arXiv 2412.16642
- LLMZip, arXiv 2306.04050
- DZip, arXiv 1911.03572
- RAS, arXiv 2511.04684
- DRAC, DOI 10.1007/s40747-021-00455-1

Uzay standartları ve veriler

- CCSDS 121.0-B-3, 123.0-B-2, 124.0-B-1 (public.ccsds.org)
- POCKET+ (DOI 10.1109/ACCESS.2024.3374227; Evans vd., SmallSat 2022)
- ESA-ADB, arXiv 2406.17826
- OPSSAT-AD, arXiv 2407.04730
- Telemanom (Hundman vd., KDD 2018, arXiv 1802.04431)
- Loghub, arXiv 2008.06448
- LogShrink, arXiv 2309.09479

CAN bus

- ROAD, arXiv 2012.14600
- can-train-and-test, arXiv 2308.04972
- CAN-BERT, arXiv 2210.09439
- SecCAN, arXiv 2505.14924
- Shreejith grubu, arXiv 2401.12234 ve 2401.12240
- Guerra vd., arXiv 2408.17235
- Marchetti ve Stabili, DOI 10.1109/IVS.2017.7995934
- UNECE R155

Sertifikasyon ve determinizm

- EASA AI Concept Paper Issue 2 (2024)
- FAA AC 20-152A; DOT/FAA/TC-19/46
- Geyer vd. (Airbus), arXiv 2401.08225
- Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference" (2025)
- ACETONE, DOI 10.4230/LIPIcs.ECRTS.2022.3
- QEBVerif (CAV 2023)
- ISO/PAS 8800:2024

Rakip platformlar

- Siracusa / Deeploy, arXiv 2408.04413
- HLSTransform, arXiv 2405.00738
- TALOS-V2 (github.com/Luthiraa/TALOS-V2)
- ESP32 ve STM32 llama2.c portları (README'ye göre)
- Xilinx DS181 (Artix-7 datasheet)